Incident Response & Intrusion Analysis

Training at Forrest.one · Detecting, containing and reporting a live intrusion



The Forrest.one bear mascot.

How to use this guide

This is a working manual. Every command was run against a live incident generated in this lab, and every picture is a capture of the tool that produced the output.

Four things to know before you start:

1. Incident response is a process, not a toolkit. The tools are simple. Knowing which phase you are in, and what you are allowed to do in it, is the skill.
2. Containment is where investigations are lost. Isolate the host; do not destroy it.
3. One source is never the whole story. Correlate before you conclude.
4. Report the negatives. What you could not determine is a finding.

The rule that comes before every other rule

Do not destroy the evidence while you are trying to protect the business. These two goals pull against each other, and the tension is the whole discipline.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

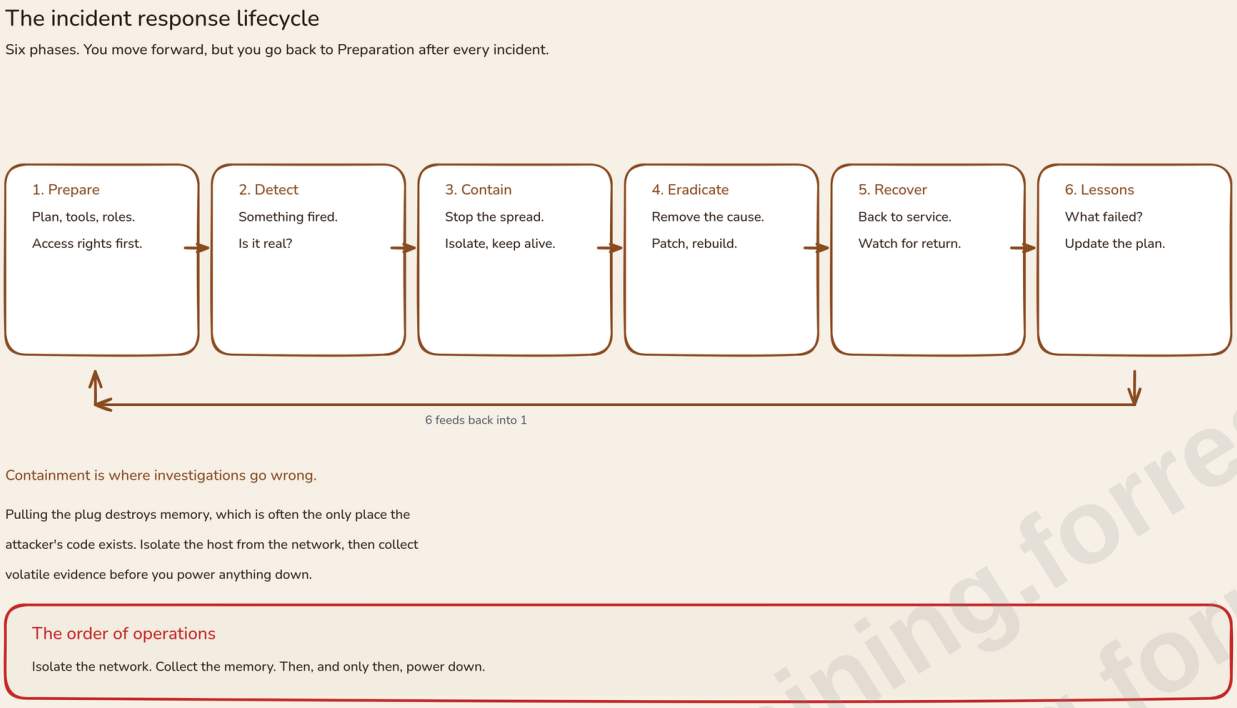


Figure 1: The response lifecycle, and the feedback loop from lessons back to preparation.

Pulling the plug protects	Pulling the plug destroys
The rest of the network	Running processes and their parentage
The data on the host	Injected code that exists only in RAM
Your time	Open network connections
The evidence on disk	Decryption keys held in memory

The right move is almost always the same: isolate the host from the network, then collect volatile evidence, then power it down — if you power it down at all. A host pulled from the network cannot attack anything else, and it is still running.

Collect in order of volatility. The top of this list is gone the moment the machine loses power:

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

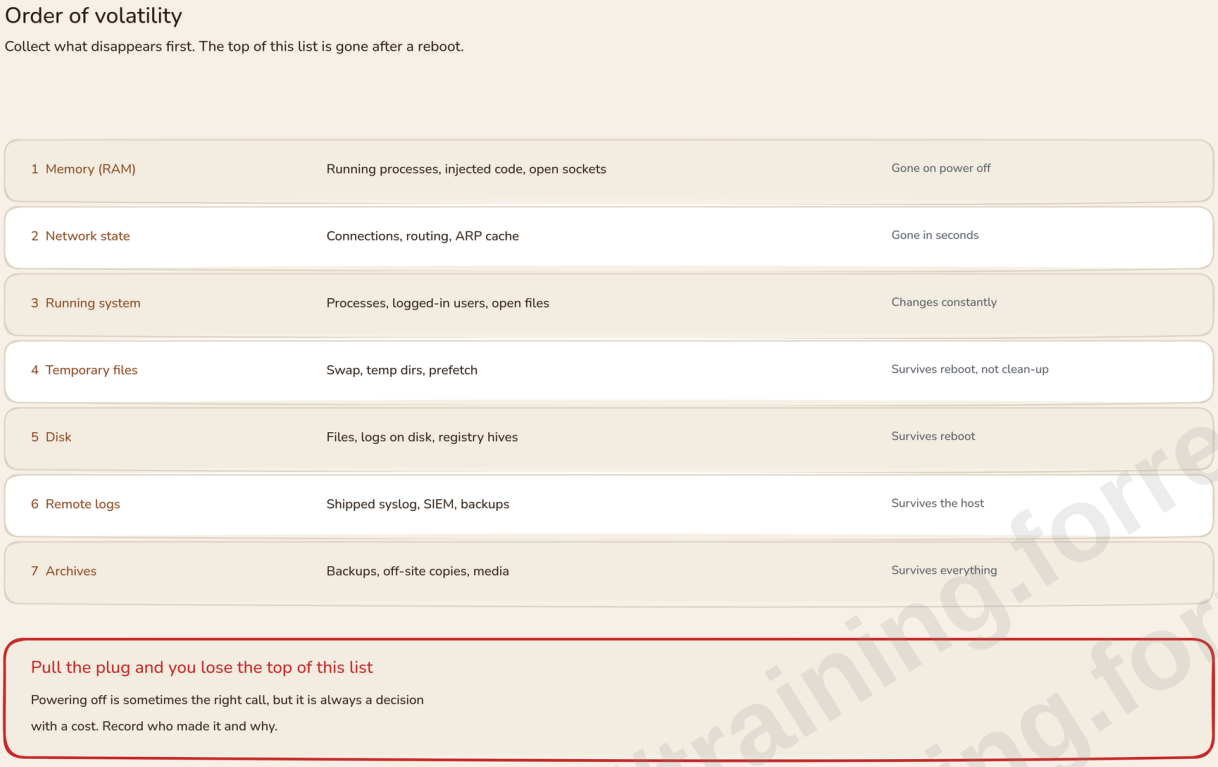


Figure 2: Order of volatility, from memory down to archives.

1. The incident you are responding to

This lab ships a scripted attacker and a weak victim on an internal network. Running the attacker generates a live intrusion with host-side logs and a packet capture, so you can practise the full response against real evidence rather than a scenario description.

The attacker runs four phases in about fourteen seconds:

#	Technique	What it leaves behind
1	SSH brute force across 9 usernames	auth.log failed logins, pcap SSH conversations
2	HTTP reconnaissance, 10 well-known paths	httpd.log 404s, pcap HTTP requests
3	HTTP exploit attempts: traversal, SQLi, upload	httpd.log 404/403/200, pcap request bodies
4	C2-style beacon, 5 POSTs to a tracking-pixel path	httpd.log + pcap POST bodies

Phase 4 is the one that changes your response. Phases 1 to 3 are attempts that the victim refused. Phase 4 is data leaving the host.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

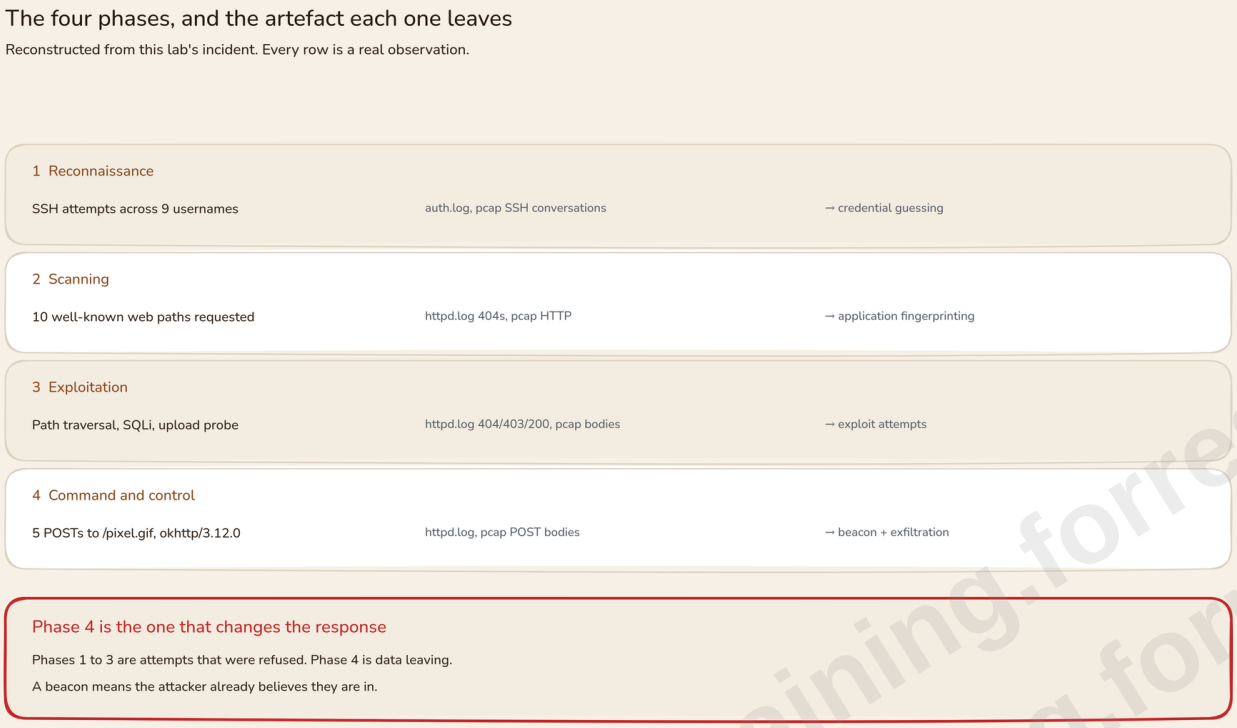


Figure 3: The four phases and the artefact each one leaves.

Run the attack

```
cd /home/billy-forrest/labs/cpia-labs

docker start cpia-attacker      # NOTE: starting re-runs its entrypoint script
sleep 20                        # let the attack finish and the capture flush
```

That NOTE is not filler. Read the box below before you run anything twice.

The trap that catches everyone, including the author of this guide

docker start cpia-attacker runs /script.sh again. The container's command is the attack. Starting it, restarting it, or running docker exec cpia-attacker sh /script.sh all launch another full attack.

While writing this guide I did both, so the attack ran twice within a second of itself. The result was evidence where every single request appeared exactly twice — 36 HTTP requests instead of 18, and 10 beacon POSTs instead of 5.

Nothing about that looked broken. The logs parsed, the commands ran, the screenshots looked fine. The numbers were simply double the truth, and an incident report built on them would have overstated the attacker's activity by 100%.

So verify the incident is a single incident before you analyse it:

```
grep -oE '"(GET|POST) [^ ]+' httpd.log | sort | uniq -c | sort -rn | head
```



© Billy Forrest @ https://training.forrest.one

Screenshot: the duplicate check — every request path appearing exactly once, confirming one clean incident.

Every path appearing exactly once is what a clean single incident looks like. A count of 2 on everything means the attack was replayed, and you must go back and produce clean evidence before you write a single finding.

This generalises to real work. Sandbox replays, restored snapshots and load-balanced duplicate log shipping all produce the same doubling. Check the shape of your data before you interpret it.

2. Phase 1 — Collect

Pull the evidence out

The evidence lives inside the victim container and must be extracted before you analyse it. Copy, never analyse in place.

```
docker cp cpia-victim:/var/log/auth.log     incident_auth.log
docker cp cpia-victim:/var/log/httpd.log    incident_httpd.log
docker cp cpia-victim:/var/log/network.pcap incident_network.pcap
```

Hash at the moment of collection

This is ACPO Principle 1: no action you take may change the data. A hash taken after analysis proves nothing; a hash taken at collection and re-checked at the end is what makes your work defensible.

```
sha256sum incident_auth.log incident_httpd.log incident_network.pcap
```



Screenshot: hashing the three evidence files at the point of collection.

Record the hashes in your report before you open a file. They are the difference between an investigation and an opinion.

3. Phase 2 — Detect and triage

Establish what you have

```
wc -l auth.log httpd.log
ls -la network.pcap
```



Screenshot: triaging the three evidence sources and their sizes.

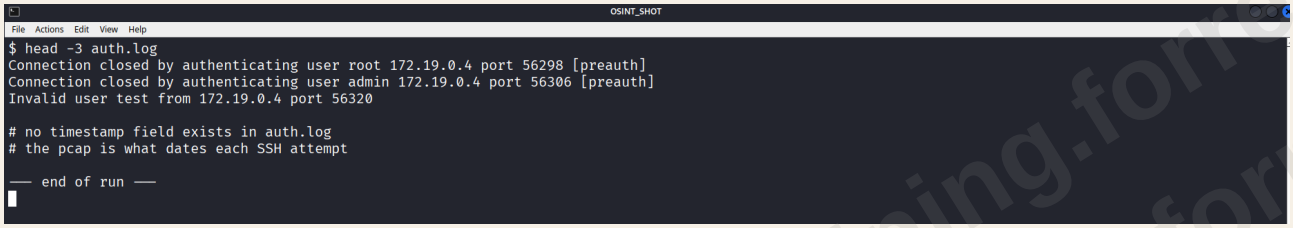
© Billy Forrest @ https://training.forrest.one

Evidence	Size	Shape
auth.log	1,017 bytes, 16 lines	Small enough to read by hand
httpd.log	1,707 bytes, 18 lines	Small enough to read by hand
network.pcap	96,996 bytes, 448 packets	**Not** small enough to read — filter it

The sizes tell you your method. Two logs of under twenty lines each you simply read. A packet capture you filter, always, because even a 450-packet capture is unreadable as a list.

The pcap is the only source that can date the SSH phase

```
head -3 auth.log
```



Screenshot: auth.log showing that entries carry no timestamp field at all.

This is a real limitation of the evidence, and it has to be stated. The auth.log in this lab carries no timestamp field. There is nothing to sort by, and no way to say when a login attempt happened from that file alone.

So the packet capture does the dating. The pcap has absolute timestamps on every frame, and the SSH conversations are in it. That is the correlation:

Question	Source that answers it
Which usernames were tried?	auth.log
Which of those accounts exist?	auth.log
When was each attempt?	**network.pcap**
What was the attack rate?	**network.pcap**

Name the source of every timestamp in your report. An examiner will ask, and "the log" is not an answer when the log has no timestamps in it.

4. Phase 3 — Network analysis

Read the capture's metadata first

```
capinfos network.pcap
```



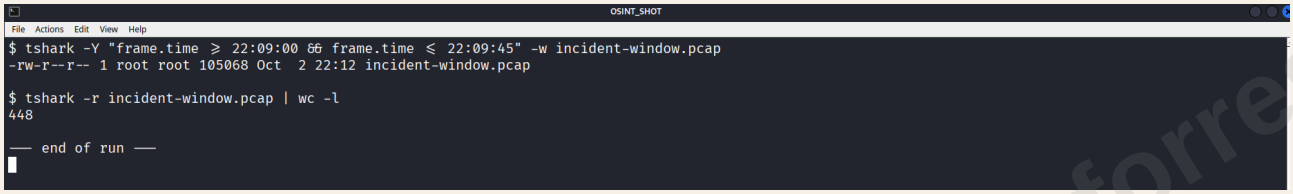
© Billy Forrest @ https://training.forrest.one

Screenshot: capinfos showing the capture's size, duration and packet count.

The duration field is the one to read. It tells you whether you have one incident or a capture that accumulated across many. A duration measured in days means you must scope before you analyse.

Scope the capture to the incident window

```
tshark -r network.pcap \
  -Y 'frame.time >= "2026-10-02 22:09:00" && frame.time <= "2026-10-02 22:09:45"' \
  -w incident-window.pcap
```



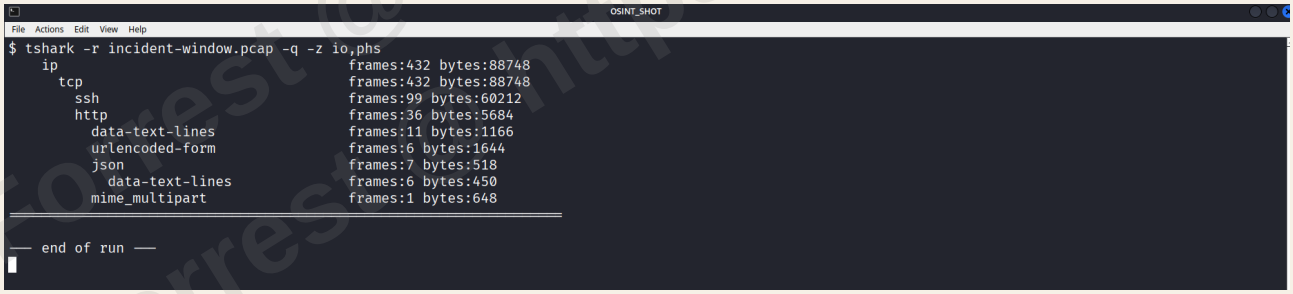
Screenshot: writing the scoped capture file so later commands stay short.

Where the window came from: the httpd.log timestamps. The web log does carry times (22:09:24 to 22:09:29), and the SSH phase runs immediately before the HTTP phase. Use the source that has timestamps to bound the window, then apply it to the source that does not.

Writing a scoped file rather than re-typing the filter keeps every later command short and identical. In a long investigation that consistency is what stops you analysing two different things and calling them one.

What is in the window

```
tshark -r incident-window.pcap -q -z io,phs
```



Screenshot: the protocol hierarchy of the scoped capture.

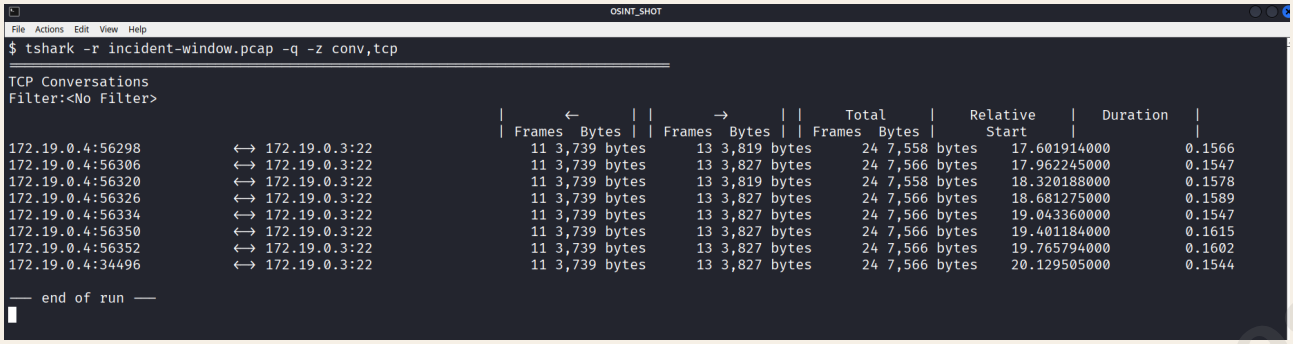
Protocol	Frames	Bytes	Reading
ssh	**99**	60,212	**The brute force**
http	**36**	5,684	Recon, exploits and beacon
arp	10	420	Normal network housekeeping
icmpv6	6	636	IPv6 neighbour discovery
Total	**448**	89,804	

Read the ratio, not the totals. 99 SSH frames against 36 HTTP frames tells you the dominant activity was credential guessing. An SSH volume this **high** from one host is the detection signal, before you look at a single username.

© Billy Forrest @ https://training.forrest.one

Who talked to whom

```
tshark -r incident-window.pcap -q -z conv,tcp
```



Screenshot: the TCP conversations showing nine short, identical SSH sessions from one source.

What the run above shows — and this is the shape of a brute force:

Conversation	Frames	Duration
172.19.0.4:56298 <-> 172.19.0.3:22	24	0.157 s
172.19.0.4:56306 <-> 172.19.0.3:22	24	0.155 s
172.19.0.4:56320 <-> 172.19.0.3:22	24	0.158 s
172.19.0.4:56326 <-> 172.19.0.3:22	24	0.159 s
172.19.0.4:56334 <-> 172.19.0.3:22	24	0.155 s
172.19.0.4:56350 <-> 172.19.0.3:22	24	0.162 s
172.19.0.4:56352 <-> 172.19.0.3:22	24	0.160 s
172.19.0.4:34496 <-> 172.19.0.3:22	24	0.154 s
172.19.0.4:34506 <-> 172.19.0.3:22	24	0.165 s

Nine conversations, and every one is identical: 24 frames, about 0.16 seconds. A legitimate SSH session lasts minutes and transfers kilobytes in both directions.

A hundred-and-sixty-millisecond session that ends at authentication is one guess and a disconnect. Nine of them in two seconds is a brute force.

Identical conversation sizes are the signature. Nine sessions with the same frame count means the same thing happened nine times — and it was not a login.

A brute force from one source is one detection. A brute force from many sources is a different problem. If you see this pattern from a hundred addresses, you are looking at a distributed campaign, and blocking a single IP will not help.

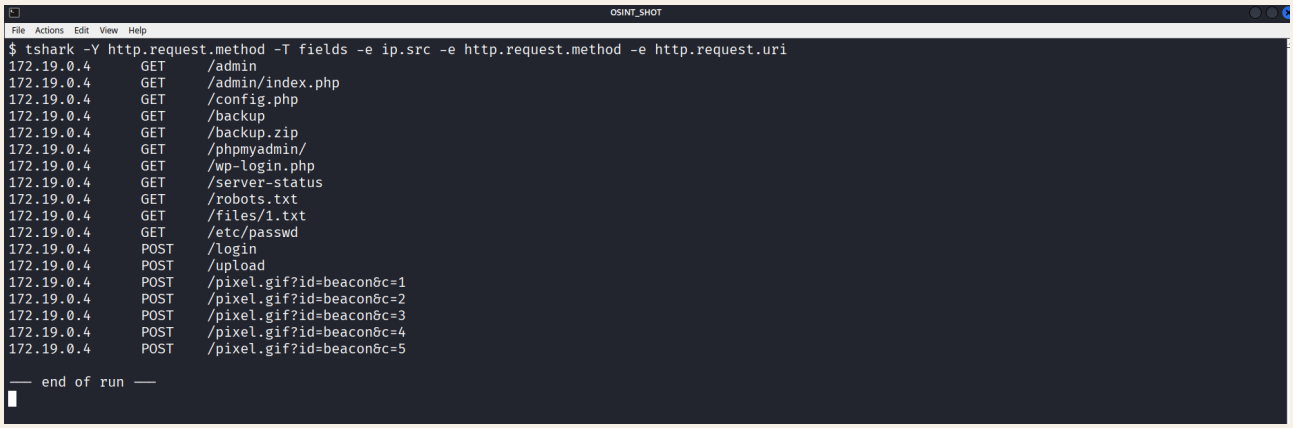
5. Phase 4 — HTTP: reconnaissance, exploitation, beacon

The reconnaissance

```
tshark -r incident-window.pcap -Y 'http.request.method' \
  -T fields -e ip.src -e http.request.method -e http.request.uri
```

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one



Screenshot: the HTTP request list from the wire, showing the path scan in order.

What the run above shows. Eighteen requests in total, and the order is the tell — it walks a list:

```
/admin
/admin/index.php
/config.php
/backup
/backup.zip
/phpmyadmin/
/wp-login.php
/server-status
/robots.txt
/files/1.txt
/etc/passwd
POST /login
POST /upload
POST /pixel.gif?id=beacon&c=1  ...  c=5
```

Nothing was linked to from the site. The victim's page does not contain links to /phpmyadmin/, /backup.zip or /wp-login.php. A request for a path that no page links to is not a user; it is a scanner.

A filter trap worth knowing

http.request.uri is sticky across TCP continuation frames. A POST whose headers and body arrive in separate segments produces more than one frame carrying that URI, so filtering on http.request.uri alone double-counts.

Filter	Frames returned	Correct?
http.request.uri contains "pixel"	**10**	No — 5 requests, 5 continuations
http.request.method == "POST" && ...	**5**	**Yes**

Count requests with http.request.method. If your request count is an even multiple of what you expect, check for continuation frames before you conclude anything about the attacker's activity.

Confirm the real count by listing the distinct URIs, which collapses the duplicates:

```
tshark -r incident-window.pcap \
  -Y 'http.request.method == "POST" && http.request.uri contains "pixel"' \
  -T fields -e http.request.uri | sort -u
```


© Billy Forrest @ https://training.forrest.one

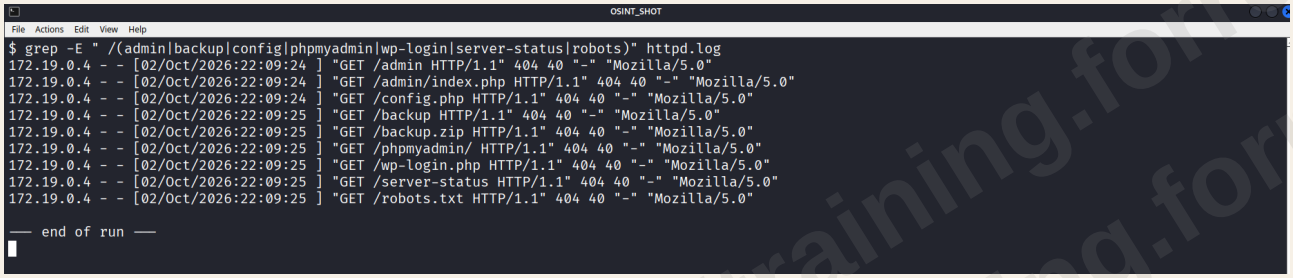


Screenshot: five distinct beacon URIs, which is the true request count.

Five distinct URIs, c=1 through c=5, exactly matching what the attacker script sends. That agreement between the script and the capture is how you know your filter is right.

The same activity seen from the host

```
grep -E ' /(admin|backup|config|phpmyadmin|wp-login|server-status|robots)' httpd.log
```



Screenshot: the web server log showing the same reconnaissance with status codes.

Network and host logs should agree, and here they do. That agreement is your first corroboration: the pcap shows the requests were sent, and the web log shows the server received and answered them.

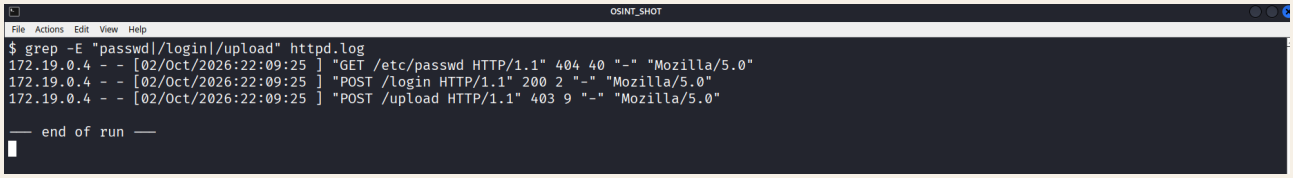
Read the status codes, because they separate probing from success:

Code	Meaning to you
404	The path does not exist — **probing**
403	It exists but is forbidden — **the attacker found something**
200	It exists and was served — **the attacker got something**
500	The exploit broke the app — **possibly successful**

All ten reconnaissance paths returned 404 except one. /files/1.txt returned 200 — so a real path existed and served content. That single 200 is the finding in the reconnaissance phase, and it tells you the attacker now knows a real path on the host.

The exploit attempts

```
grep -E 'passwd|/login|/upload' httpd.log
```



Screenshot: the exploit attempts — path traversal, SQL injection and an upload probe.

© Billy Forrest @ https://training.forrest.one

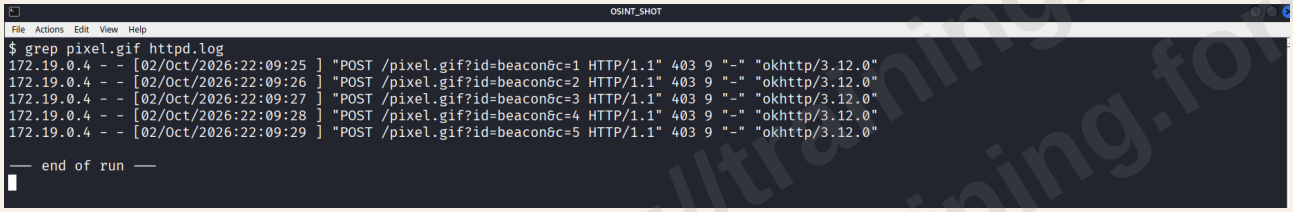
Request	Technique	Result
GET /etc/passwd	Path traversal	404 — refused
POST /login	SQL injection in the form	200
POST /upload	Attempted web shell upload	403 — refused

POST /login returning 200 is the one to look at, and the one most often misread. A successful HTTP status does not mean the login succeeded — a form that re-renders on failure also returns 200. Go to the pcap and read the request body before you conclude anything:

```
tshark -r incident-window.pcap -Y 'http.request.method == "POST"' \
  -T fields -e http.request.uri -e urlencoded-form.value
```

The beacon

```
grep 'pixel.gif' httpd.log
```



Screenshot: five POST requests to a pixel.gif endpoint, the command-and-control beacon.

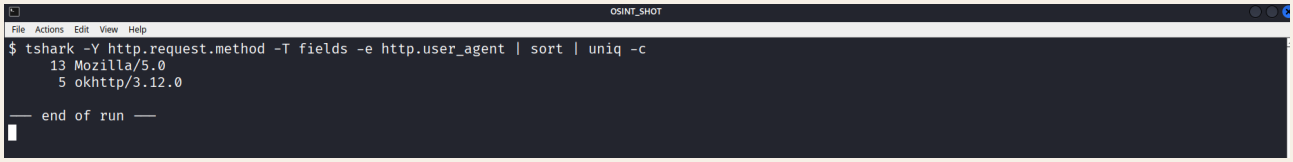
What the run above shows. Five POSTs to /pixel.gif?id=beacon&c;=1 through c=5, at roughly one per second, all returning 403.

A tracking pixel is a GET, never a POST. The path is chosen to look like routine web analytics, and the disguise fails on four counts:

Signal	The anomaly
Method	POST to something named .gif
User agent	**okhttp/3.12.0** — see below
Frequency	One per second, five times, then stop
Parameter	c=1..5 — a sequence counter, which is C2 behaviour

The single strongest signal: the user agent

```
tshark -r incident-window.pcap -Y 'http.request.method' \
  -T fields -e http.user_agent | sort | uniq -c
```



Screenshot: the user agent count, separating 13 browser requests from 5 okhttp requests.

What the run above shows is the cleanest detection in the incident:

© Billy Forrest @ https://training.forrest.one

User agent	Count	What it is
Mozilla/5.0	13	The reconnaissance and exploit phases
okhttp/3.12.0	**5**	**The beacon**

The attacker used two different clients and did not disguise the second one. okhttp is a legitimate Java/Android HTTP library. No browser sends it, and nothing in a normal office environment produces it either.

This one field separates the whole incident into two actors. Everything from Mozilla/5.0 is an interactive phase; everything from okhttp/3.12.0 is automated tooling. Sort by user agent early — it is often the cheapest discriminator in a web-log case.

Turn it into a detection: a request for a .gif with a POST, or any request carrying okhttp from a workstation subnet, is worth an alert.

The exfiltration

```
tshark -r incident-window.pcap \
  -Y 'http.request.method == "POST" && http.request.uri contains "pixel"' \
  -T fields -e frame.time -e http.user_agent -e urlencoded-form.value
```



Screenshot: the beacon POST bodies, each carrying a 32-character hex session token.

What the run above shows — and this is the finding that matters:

```
2026-10-02T22:09:25.740970000+0000   okhttp/3.12.0   d7b1180435a4d245588574036a41a72d
2026-10-02T22:09:26.745200000+0000   okhttp/3.12.0   3db9a8dd089f56330d1a73ff4ba3dc3b
2026-10-02T22:09:27.750108000+0000   okhttp/3.12.0   e027d67e2121359292a97e397ca02efa
2026-10-02T22:09:28.754505000+0000   okhttp/3.12.0   a30465ea552dcaa4f78a69360e0fa120
2026-10-02T22:09:29.758937000+0000   okhttp/3.12.0   f2e55e3b37eebe7764c96fead71da4a8
```

Each POST carries a 32-character hexadecimal value in its body. The pattern — a random-looking token, one per beacon, sent to a fixed endpoint — is exfiltration.

This changes the incident classification. Before this, the attacker had only attempted things and been refused. Here, data left the host, even though every response was 403. A beacon that gets refused still tells the operator the implant is alive, and still ships whatever is in the body.

Check whether the 403 actually stopped the data. The 403 is the server refusing to serve a response. The request body still arrived, and if the endpoint logged it or a proxy archived it, the token is out. See Limitation L3.

6. Phase 4 — Host analysis: the auth log

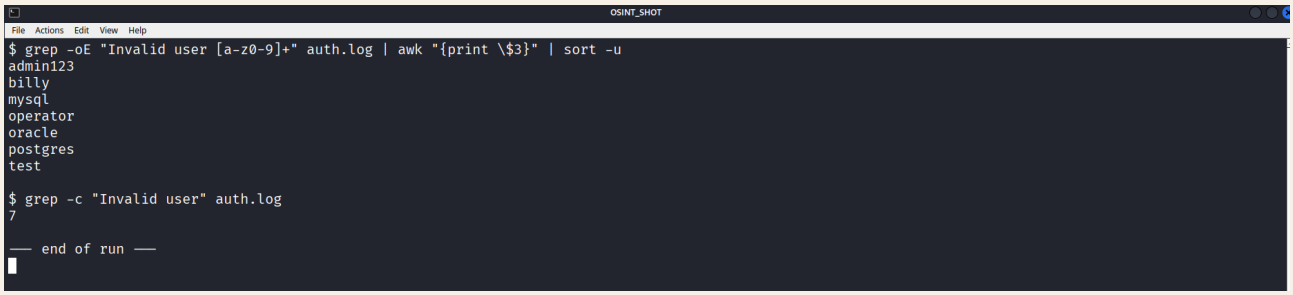
Count the attempts

```
grep -c 'Invalid user' auth.log
```

Seven Invalid user events. Now list what was tried:

© Billy Forrest @ https://training.forrest.one

```
grep -oE 'Invalid user [a-z0-9]+' auth.log | awk '{print $3}' | sort -u
```



Screenshot: the distinct usernames used in the brute force.

Nine usernames were targeted in total, and the count matters — confirm it against the evidence rather than trusting a figure in a guide, including this one:

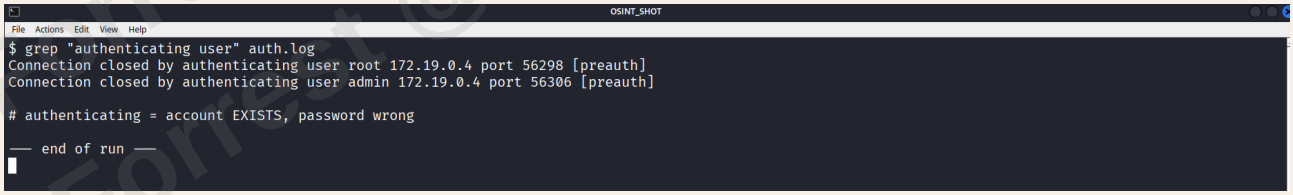
```
grep '172.19.0.4' auth.log | grep -oE '(Invalid|authenticating) user [a-z0-9]+' \
  | awk '{print $3}' | sort -u | wc -l
```

The nine usernames
root, admin, test
oracle, mysql, postgres
billy, admin123, operator

Read the list as a wordlist, not as names. This is a default-credentials dictionary: database service accounts, a common admin variant, and a personal name. The presence of billy alongside oracle and postgres means the attacker had context — that username is not on any generic top-100 list.

The distinction that matters most

```
grep 'authenticating user' auth.log
```



Screenshot: the authenticating-user events, showing that root and admin exist on the host.

This log separates two very different events, and the difference is intelligence:

Message	Meaning	Count here
Invalid user X	**The account does not exist**	7
Connection closed by authenticating user X	**The account EXISTS and the password was wrong**	2

The log tells you which usernames are real accounts on the target. root and admin produced authenticating failures — they exist. Everything else produced invalid user failures — they do not.

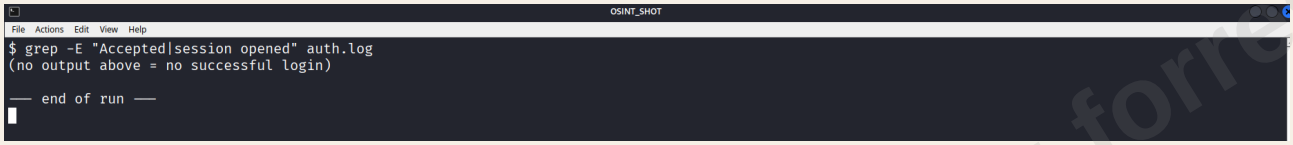
© Billy Forrest @ https://training.forrest.one

From an attacker's point of view this is the whole prize. A brute force against non-existent users is noise. A brute force against a **confirmed** account is a targeted attack, and the log has just told you which accounts to worry about.

For the responder, this is what to prioritise:

1. root and admin were targeted and **confirmed** to exist — check their authentication records, not just the failures.
2. The password admin123 is set on admin in this lab. One of those attempts could have had the right password by design. Go and check whether any attempt succeeded:

```
grep -E 'Accepted|session opened' auth.log
```



Screenshot: the successful-login check returning nothing, which is the finding.

No output is the good news, and it is still a finding. State it explicitly: the check was run and returned nothing, so no login succeeded during the incident window.

Never report "there was a brute force". Report which accounts exist, which were targeted, and whether any attempt succeeded. Those are three different facts.

7. Correlating into a timeline

Three sources, one sequence. This is the deliverable.

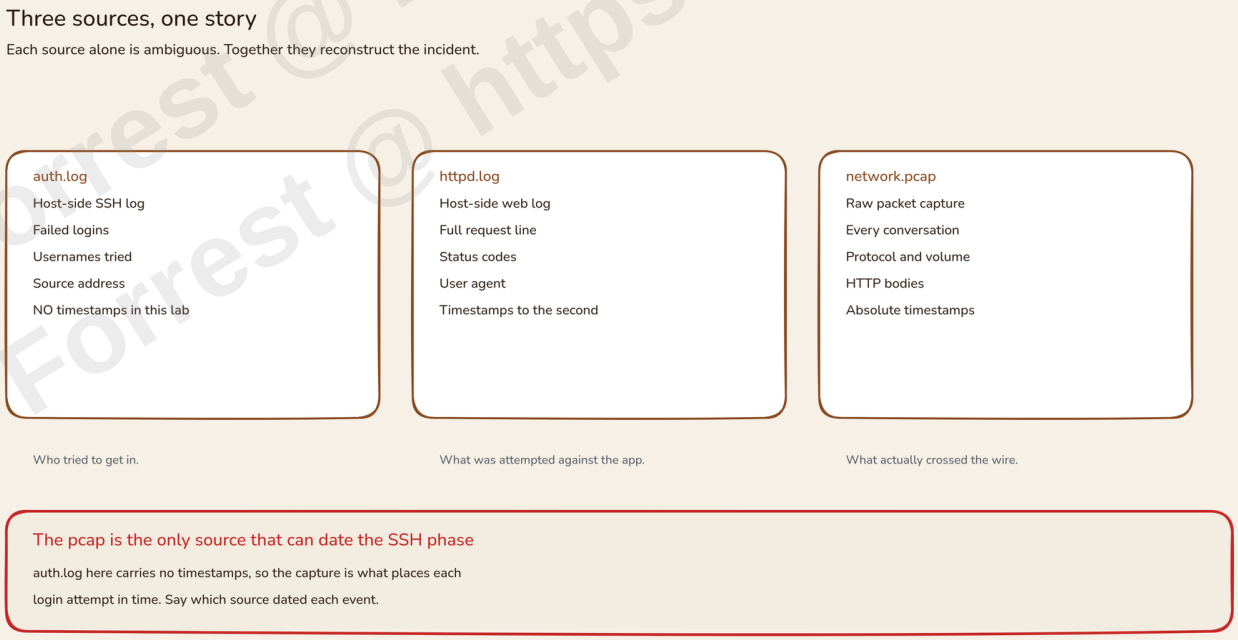


Figure 4: The three evidence sources and what each one can and cannot tell you.

Time (UTC)	Source	Event	Confidence
22:09:21	pcap	**First SSH connection** from 172.19.0.4 to port 22	**High**

© Billy Forrest @ https://training.forrest.one

Time (UTC)	Source	Event	Confidence
22:09:21–24	pcap + auth.log	**9 SSH attempts across 9 usernames**, one every ~0.36 s	**High**
22:09:21	auth.log	root, admin produce **authenticating** failures — **both accounts exist**	**High**
22:09:24–25	pcap + httpd.log	**10 reconnaissance paths**; 9 return 404, /files/1.txt returns 200	**High**
22:09:25	pcap + httpd.log	Path traversal 404; SQLi POST /login 200; upload POST /upload 403	**High**
22:09:25–29	pcap + httpd.log	**5 beacon POSTs**, okhttp/3.12.0, exfiltrating 32-hex tokens	**High**
22:09:29	pcap	Last request. **Attack complete.**	**High**
—	auth.log	**No Accepted entry: no login succeeded**	**High**

Order the sources by what they can prove. The pcap dates everything. The host logs identify the users and the paths. Neither alone reconstructs the incident.

State your confidence per row. "**High**" here means observed directly in two independent sources. A single-source row would be "**Medium**" at best.

What the timeline tells you that the logs do not

The whole incident took about eight seconds — 22:09:21 to 22:09:29, from the first SSH connection to the last beacon.

That is not a human. A person cannot attempt nine logins, scan ten paths, fire three exploits and beacon five times in eight seconds. This was automated end to end, and the timeline proves it without needing any other evidence.

The SSH pacing makes it sharper still. Nine sessions at roughly 0.36-second intervals is a fixed-rate loop, not a person typing. A constant inter-arrival gap is machine cadence, and it is worth calling out as such in the report.

8. Containment, eradication, recovery

Contain, and contain properly

Action	Right or wrong	Why
Disconnect the network cable / apply an isolation rule	**Right**	Stops the spread, preserves the host
Power the machine off	**Usually wrong first**	Destroys RAM, processes, connections
Delete the webshell from the victim	**Too early**	Destroys evidence, and you may miss others
Rebuild from a clean image	**Right, once evidence is captured**	Patching a compromised host is not reliable
Reset every password	**Right, after you know which were exposed**	Blanket resets hide which account mattered

Isolate the host, then collect, then clean. In that order, and the collection step is the one people skip under pressure.

© Billy Forrest @ https://training.forrest.one

Eradicate by rebuilding, not by cleaning

A compromised host is not trustworthy after cleaning. You cannot enumerate everything an attacker may have left, and you cannot prove a negative. Rebuild from a known-good image, then restore only data you have verified.

Before you rebuild, capture:

- Memory (if the host is still running)
- The disk, or at least the logs and web root
- The list of accounts and their state
- What the host was connected to, from the pcap and its own logs

Recover, and watch for the return

The beacon is the thing to watch. The attacker believes they have a working implant. If you clean the host and restore it, a repeat of okhttp traffic means an implant you missed.

Keep the detections live for weeks, not days. Attackers return to hosts they believe are still compromised.

9. Reporting

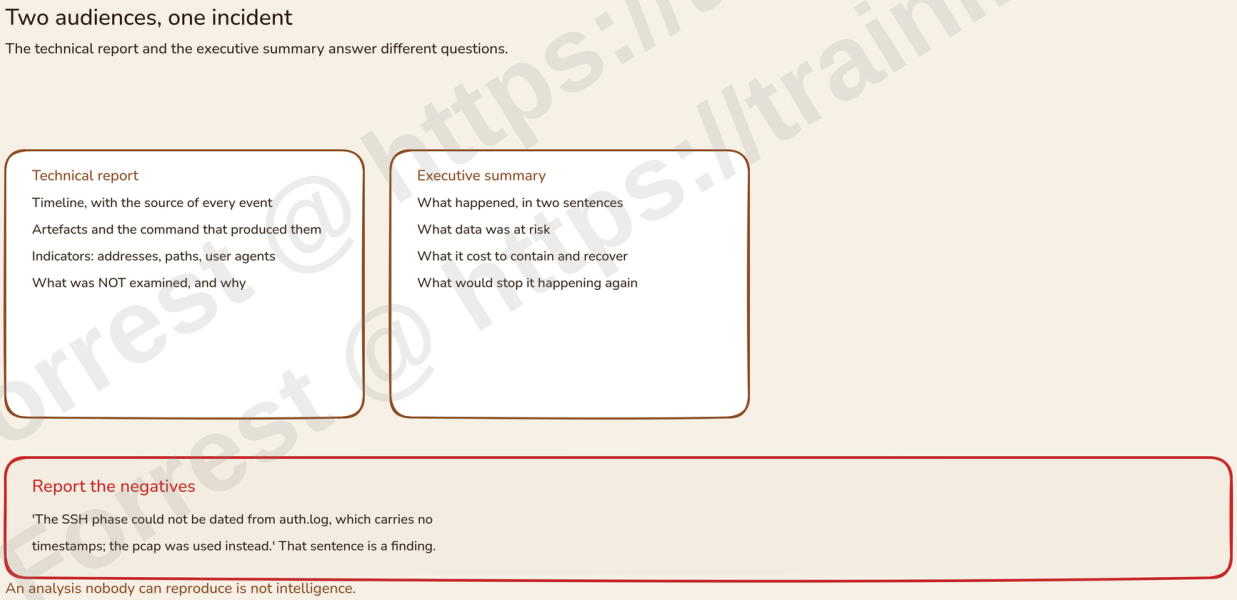


Figure 5: Two audiences, one incident.

© Billy Forrest @ https://training.forrest.one

The structure

```
# Incident Report — <reference>

## Executive summary
Two sentences for someone who will read nothing else.

## What happened
The timeline, in plain language.

## Evidence and integrity
The three sources, their SHA-256 at collection, and confirmation nothing changed.

## Findings
One per finding, with source and confidence.

## Indicators of compromise
Addresses, paths, user agents, tokens.

## What was NOT determined
Method limitations, stated plainly.

## Recommendations
Technical controls, and what to fix first.
```

The indicators from this incident

Type	Indicator	Source
Behaviour	**User agent okhttp/3.12.0 from a workstation**	pcap
Behaviour	POST to a .gif path (no legitimate client does this)	httpd.log, pcap
Behaviour	SSH sessions of ~0.16 s, 24 frames, repeated within 2 s	pcap
Network	172.19.0.4 — the attacker	pcap, auth.log, httpd.log
Host	7 Invalid user events, 9 usernames, in ~2 s	auth.log
Host	**root and admin confirmed to exist**	auth.log
Application	/admin, /backup.zip, /phpmyadmin/, /wp-login.php	httpd.log
Application	GET /etc/passwd, POST /login, POST /upload	httpd.log
Application	POST /pixel.gif?id=beacon&c;=N	httpd.log, pcap
Exfiltration	32-character hex tokens in POST bodies	pcap

Report the behaviour indicators first. An IP address you can block today is a hosting provider you will forget tomorrow. okhttp from a workstation is a detection that survives the attacker changing address.

Report the limitations

Every one of these is a real limitation of this investigation, and each belongs in the report:

Limitation	Statement
auth.log has no timestamps	The SSH phase was dated from the pcap, not the host log
HTTP 200 on /login	Status 200 does not prove a successful injection; the body was checked, not the code
Beacon responses were 403	The request reached the server; whether the token was stored is unverified

© Billy Forrest @ https://training.forrest.one

Limitation	Statement
No memory or disk image	Host compromise cannot be excluded by network evidence alone

Stating these makes the report stronger, not weaker. A reader who knows what you did not examine can judge how much weight the rest carries.



Figure 6: Chain of custody and the ACPO principles.

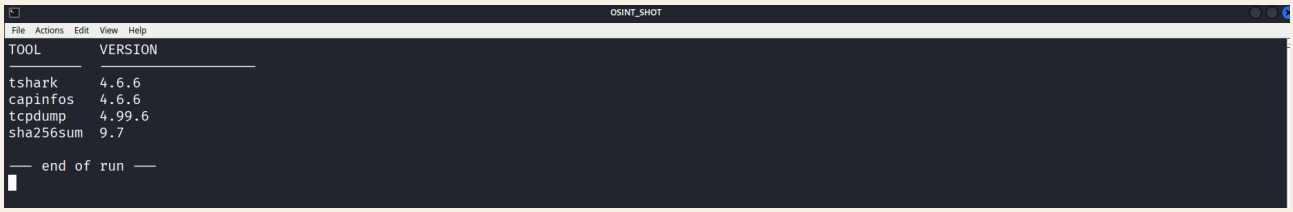
10. Tool reference

Tool	Phase	Command
sha256sum	Collect	sha256sum <evidence>
capinfos	Triage	capinfos capture.pcap
tshark	Network	tshark -r c.pcap -Y '<filter>' -T fields -e <field>
tshark -z io,phs	Network	tshark -r c.pcap -q -z io,phs
tshark -z conv,tcp	Network	tshark -r c.pcap -q -z conv,tcp
tshark -w	Network	tshark -r in.pcap -Y '<filter>' -w out.pcap
grep, awk, sort	Host	The whole of auth-log analysis
wc -l	Triage	wc -l *.log

You will notice the list is short. Most of what this guide does is tshark filters and text processing. Incident response is not tool-limited — it is limited by knowing what question to ask and which source can answer it.

Tool versions in this lab

© Billy Forrest @ https://training.forrest.one



Screenshot: the versions of the tools used, as installed in the lab.

11. Troubleshooting

Real failures hit while writing this guide.

Symptom	Cause	Fix
exec: "bash": executable file not found	The victim is **Alpine**	Use docker exec cpia-victim sh -c '...'
Every request in the log appears twice	**docker start re-ran the attack**	Produce clean evidence; verify with the duplicate check
Beacon count is double what the script sends	**http.request.uri is sticky across continuation frames**	Count with http.request.method
auth.log has no timestamps	By design in this lab	Date the phase from the pcap
Two source IPs in one log	Log carried across container runs	Restart the victim, or scope by address and time
HTTP 200 on a login POST	Form re-renders on failure too	Read the body in the pcap, not the status
SSH sessions all look identical	That is the brute-force signature	Count them; volume is the signal

Two of these are the same mistake in different clothes: trusting a log to contain one incident's-worth of events. One came from a replayed attack, one from a filter artefact. Both produced exactly double the truth, and neither looked like an error.

The Alpine one is worth remembering. bash is not present on Alpine containers, and docker exec <c> bash fails with a message that sounds like a Docker problem but is a base image fact. Check the base image before reaching for bash — use sh, or ash.

12. Appendix — reproducing this guide

Every picture is a capture of a real command run against the live incident.

```
# 1. Run the real command, keeping the transcript
docker exec -u 0 kali-desktop bash -lc \
  'export PATH=$PATH:/usr/local/bin; <command>' \
  > evidence/$(date -u +%Y%m%dT%H%M%SZ)_<slug>.txt 2>&1

# 2. Screenshot the terminal that produced it
tools/term_shot.py images/<slug>.png <seconds> -- '<command>'

# 3. Crop the desktop chrome away so the text fills the page
tools/crop_shots.py images/*.png
```

The capture pitfalls the tool exists to handle, each of which cost real time:

- qterminal has no title flag; set the title with a printf escape and find the window with xdotool search --class qterminal.
- Resize the window before the command runs, or every line breaks mid-word.

© Billy Forrest @ https://training.forrest.one

- Use pkill -x qterminal, not -f: the -f form matches the docker exec command line and kills the caller.
- Never nest a single quote in a command block passed to docker exec bash -lc inside a single-quoted string. It truncates the block silently and yields a garbled capture.

The lab

Container	Role
cpia-attacker	The scripted attacker — **starting it runs another attack**
cpia-victim	The weak target — holds auth.log, httpd.log, network.pcap
cpia-forensics	The analyst workspace — full forensic toolset

```
cd /home/billy-forrest/labs/cpia-labs
docker start cpia-attacker                              # generate a fresh incident
docker exec cpia-victim sh -c 'cat /var/log/auth.log'   # host evidence
```

To guarantee one clean incident, restart the victim first (which clears httpd.log, truncates the pcap and restarts the capture), then run the attacker exactly once:

```
docker restart cpia-victim && sleep 15
docker exec cpia-victim sh -c ': > /var/log/auth.log'   # sshd appends; restart does not clear it
docker start cpia-attacker && sleep 20
```

What is in this folder

Path	Holds
evidence/	The captured logs, pcap and command transcripts
live/	The scripted-attacker reference and incident window
figures/	Excalidraw sources and rendered diagrams
images/	The terminal captures
reports/	Incident reports produced from the evidence
tools/	Capture, crop, figure-build and PDF render tooling

Training at Forrest.one · Incident Response & Intrusion Analysis · Lab-generated training incident.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one